

metacharacters

- Shell 특수기호는 Shell 메타문자로 불리기도 한다. 쉘은 이런 특수기호들을 해석 (interpret)하여 명령을 실행한다.
- 그래서 쉘 메타문자를 잘 알아두고 일반 문자와 같이 사용하지 않도록 조심할 필요가 있다.

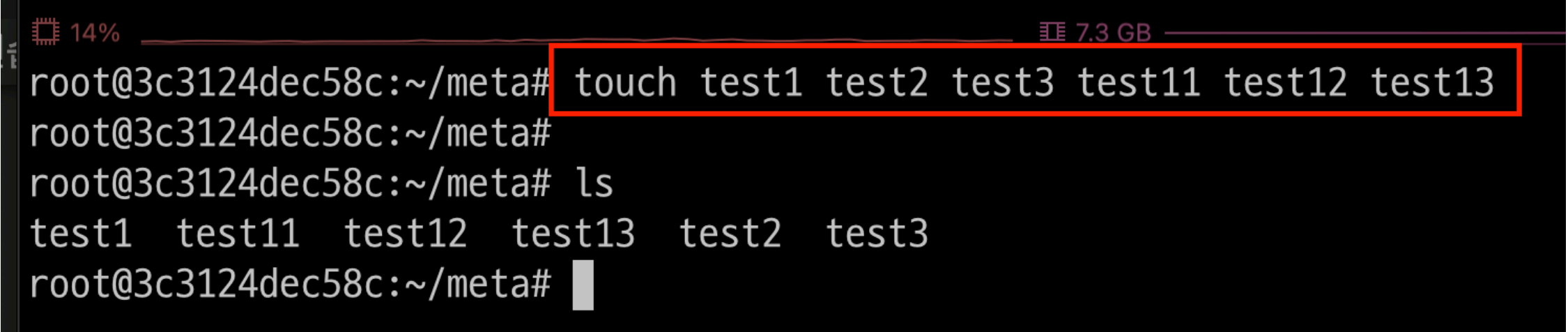
단계1: 연습용 폴더 생성

```
cd ~  
mkdir meta  
cd meta
```

```
root@3c3124dec58c:~# mkdir meta  
root@3c3124dec58c:~# cd meta  
root@3c3124dec58c:~/meta# ls -al  
total 8  
drwxr-xr-x 2 root root 4096 Feb  6 02:28 .  
drwx----- 1 root root 4096 Feb  6 02:28 ..  
root@3c3124dec58c:~/meta#
```

단계2: 연습용 파일 생성

```
touch test1 test2 test3 test11 test12 test13  
ls
```

A terminal window with a dark background. The top status bar shows a battery icon at 14%, a memory icon at 7.3 GB, and a vertical bar on the left. The terminal text shows a user at the root of a VM with IP 3c3124dec58c, in the directory ~/meta. The command 'touch test1 test2 test3 test11 test12 test13' is entered and highlighted with a red box. Subsequent commands 'ls' and the resulting output 'test1 test11 test12 test13 test2 test3' are shown.

```
root@3c3124dec58c:~/meta# touch test1 test2 test3 test11 test12 test13  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls  
test1 test11 test12 test13 test2 test3  
root@3c3124dec58c:~/meta#
```

1. File Name Extension

- 파일 이름을 확장하는 데 사용되는 메타문자들이다.

메타 문자	의미
*	문자열 wildcard
?	문자 1개 wildcard
[]	문자 집합 wildcard
{ }	문자열 집합 (comma로 분리)

단계1: *

```
ls * # 전체 파일  
ls te* # te로 시작하는 전체 파일  
ls test1* # test1로 시작하는 전체 파일
```

```
root@3c3124dec58c:~/meta# ls *  
test1 test11 test12 test13 test2 test3  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls te*  
test1 test11 test12 test13 test2 test3  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls test1*  
test1 test11 test12 test13  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta#
```

단계2: ?

```
ls test? # test + 하나의 글자 > 조회  
ls t?st1 # t + 하나의 글자 + st1 > 조회  
ls test1? # test1 + 하나의 글자 > 조회
```

```
root@3c3124dec58c:~/meta# ls test?  
test1 test2 test3  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls t?st1  
test1  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls test1?  
test11 test12 test13  
root@3c3124dec58c:~/meta#
```

단계3: []

```
# test + (1 또는 2) > 조회  
ls test[12]
```

```
# test + (1 또는 2) + (1 또는 3) > 조회  
ls test[12][13]
```

```
root@3c3124dec58c:~/meta# ls test[12]  
test1 test2  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls test[12][13]  
test11 test13  
root@3c3124dec58c:~/meta#
```

단계4: {}

```
# test + (1 또는 3) > 조회  
ls test{1,3}
```

```
# test + (1 또는 3 또는 12) > 조회  
ls test{1,3,12}
```

```
root@3c3124dec58c:~/meta# ls test{1,3}  
test1 test3  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls test{1,3,12}  
test1 test12 test3  
root@3c3124dec58c:~/meta#
```


2. Command Substitution

- 커맨드 대치는 back-quote (`) 안의 문자열을 명령어로 실행한다.

date # date 명령어 실행

d 변수에 **date** 명령어의 실행 결과 적용
d=`date`
echo \$d # 확인

```
root@3c3124dec58c:~/meta# date
Tue Feb  6 03:03:58 UTC 2024
root@3c3124dec58c:~/meta#
root@3c3124dec58c:~/meta# d=`date`
root@3c3124dec58c:~/meta#
root@3c3124dec58c:~/meta# echo $d
Tue Feb 6 03:04:06 UTC 2024
root@3c3124dec58c:~/meta#
```

3. Escape character

- 이스케이프 문자는 셸이 해석하는 것을 원하지 않을 때 사용하는 것이다.
- 우리가 입력한 그대로의 문자열이 명령의 인자로 들어가게끔!

메타문자	의미
\	Single character escape
' '	String escape (String을 그대로 자식 process에 전달)
" "	String escape (\$와 `를 expand 하고 자식 process에 전달)

단계1: \

```
ls * # 특수문자로 인식 > 전체 데이터 조회  
ls \* # 단순문자로 인식  
touch * # 특수문자로 인식 > 파일 생성 x  
touch \* # 단순문자로 인식 > 파일 생성 o
```

```
root@3c3124dec58c:~/meta# ls *  
test1 test11 test12 test13 test2 test3  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# ls \*  
ls: cannot access '*': No such file or directory  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# touch *  
root@3c3124dec58c:~/meta# ls  
test1 test11 test12 test13 test2 test3  
root@3c3124dec58c:~/meta# touch \*  
root@3c3124dec58c:~/meta# ls  
'*' test1 test11 test12 test13 test2 test3  
root@3c3124dec58c:~/meta#
```

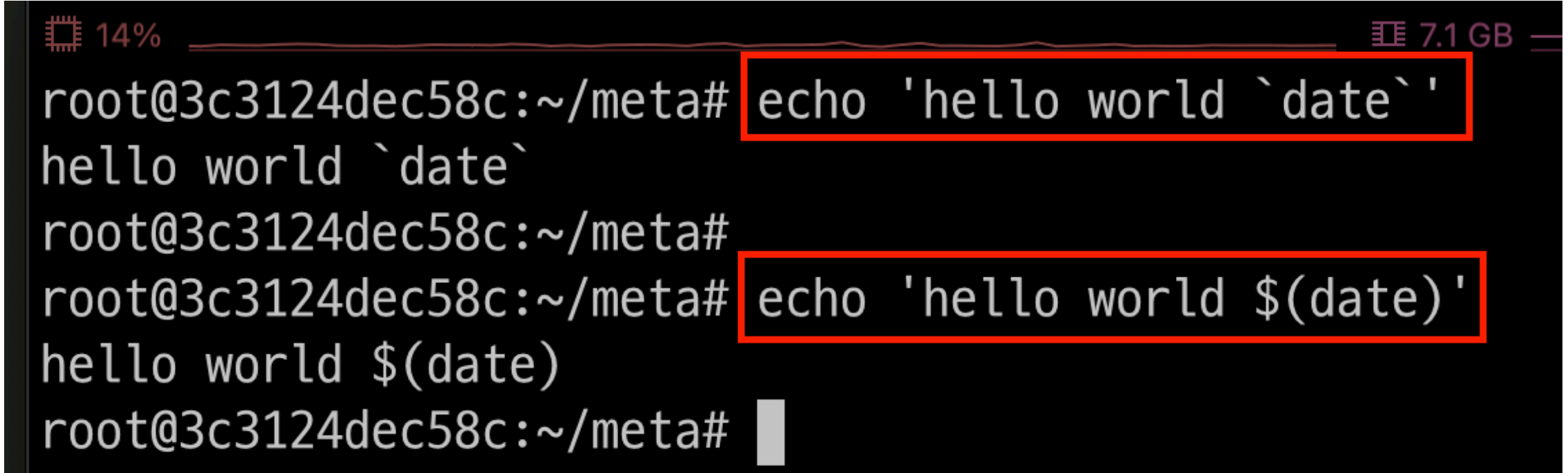
단계2: ""

```
echo "hello world `date`"  
echo "hello world $(date)"
```

```
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# echo "hello world `date`"  
hello world Tue Feb  6 03:14:01 UTC 2024  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# echo "hello world $(date)"  
hello world Tue Feb  6 03:14:13 UTC 2024  
root@3c3124dec58c:~/meta#
```

단계3: "

```
echo 'hello world `date`'  
echo 'hello world $(date)'
```

A terminal window with a dark background. The top status bar shows a battery icon at 14%, a memory icon at 7.1 GB, and a window icon. The terminal text shows a user at the root prompt in the directory ~/meta. The first command 'echo 'hello world `date`'' is entered and its output 'hello world `date`' is shown. The second command 'echo 'hello world \$(date)'' is entered and its output 'hello world \$(date)' is shown. The prompt is currently at the root@3c3124dec58c:~/meta# line.

```
root@3c3124dec58c:~/meta# echo 'hello world `date`'  
hello world `date`  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# echo 'hello world $(date)'  
hello world $(date)  
root@3c3124dec58c:~/meta#
```

4. Job control meta characters

- Subshell
 - 현재 셸 안에서 실행되는 셸로, 현재 셸의 자식 프로세스로 실행

메타 문자	의미
&	백그라운드 job
;	셸 명령 분리자
()	subshell
&&	조건부 실행 (and)
	조건부 실행 (or)

단계1: ;

```
date; pwd  
date; pwd; ls
```

```
root@3c3124dec58c:~/meta# date; pwd
```

```
Tue Feb  6 03:19:17 UTC 2024
```

```
/root/meta
```

```
root@3c3124dec58c:~/meta#
```

```
root@3c3124dec58c:~/meta# date; pwd; ls
```

```
Tue Feb  6 03:19:52 UTC 2024
```

```
/root/meta
```

```
'*'      test1      test11      test12      test13      test2      test3
```

```
root@3c3124dec58c:~/meta#
```

단계2: &&

출력: data 오류 & pwd 결과
data; pwd

출력: data 오류 > pwd 생략
data && pwd

출력: date 결과 & pwd 결과
date && pwd

```
root@3c3124dec58c:~/meta# data; pwd
bash: data: command not found
/root/meta
root@3c3124dec58c:~/meta#
root@3c3124dec58c:~/meta# data && pwd
bash: data: command not found
root@3c3124dec58c:~/meta#
root@3c3124dec58c:~/meta# date && pwd
Tue Feb  6 03:23:37 UTC 2024
/root/meta
root@3c3124dec58c:~/meta#
```


단계2: ||

```
# data 오류 발생 > pwd 실행  
data || pwd
```

```
# date 성공 > pwd 생략  
date || pwd
```

```
root@3c3124dec58c:~/meta# data || pwd  
bash: data: command not found  
/root/meta  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# date || pwd  
Tue Feb  6 03:28:35 UTC 2024  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta#
```

5. I/O 관련

메타문자	의미
	파이프 (pipe)
< <<	입력 재지정 (redirection)
> >>	출력 재지정 (redirection)
#	주석 (comment)
\$	셸 변수
!	History 기능

단계1: |

```
# 전체 변수 출력
set
# 전체 변수 중에서 OP 문자 출력
set | grep OP
# 전체 변수 중에서 OP 문자 출력의 count
set | grep OP | wc -l
```

```
TERM=xterm
UID=0
_=clear
d='Tue Feb  6 03:04:06 UTC 2024'
root@3c3124dec58c:~/meta# set | grep OP
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extquote:force_ignore:globasc
iiranges:histappend:hostcomplete:interactive_comments:progcomp:promptvars:sourcepath
OPTERR=1
OPTIND=1
SHELLOPTS=braceexpand:emacs:hashall:histexpand:history:interactive-comments:monitor
root@3c3124dec58c:~/meta# set | grep OP | wc -l
4
root@3c3124dec58c:~/meta#
```

단계2: >, >>

```
echo "hello world" > sample1
```

overwrite 발생

```
echo "good morning" > sample1
```

append 발생

```
echo "hello world" >> sample1
```

```
root@3c3124dec58c:~/meta# echo "hello world" > sample1
root@3c3124dec58c:~/meta# cat sample1
hello world
root@3c3124dec58c:~/meta# echo "good morning" > sample1
root@3c3124dec58c:~/meta# cat sample1
good morning
root@3c3124dec58c:~/meta# echo "hello world" >> sample1
root@3c3124dec58c:~/meta# cat sample1
good morning
hello world
root@3c3124dec58c:~/meta#
```

단계3: <

```
cat < sample1  
cat < sample1 | grep hello
```

A terminal window with a dark background. The prompt is 'root@3c3124dec58c:~/meta#'. The first command 'cat < sample1' is entered and highlighted with a red box. The output is 'good morning' followed by 'hello world' on the next line. The second command 'cat < sample1 | grep hello' is entered and highlighted with a red box. The output is 'hello world' on the next line, where 'hello' is highlighted in red. The prompt is shown again at the bottom.

```
root@3c3124dec58c:~/meta# cat < sample1  
good morning  
hello world  
root@3c3124dec58c:~/meta#  
root@3c3124dec58c:~/meta# cat < sample1 | grep hello  
hello world  
root@3c3124dec58c:~/meta#
```